LifeSync Test Report

Report revision date: 2026-04-16
Project: LifeSync
Repository: beyzadegirmenci/LifeSync

1. Test Responsibles

Area	Responsible
Test execution and report preparation	Local QA / automation run in current workspace
Backend unit test ownership	Backend development team
Frontend smoke flow ownership	Frontend and QA automation
Repository contributors referenced for project ownership	`Mehmet Eski`, `Kerem Elma`

2. Test Date

Activity	Date	Status
Backend unit test execution	2026-04-16	Completed
Frontend production build validation	2026-04-16	Completed
Selenium end-to-end smoke validation	2026-04-16	Completed
Test artifact recording update	2026-04-16	Completed

3. Test Configuration

Environment

Item	Value
OS	Windows (PowerShell environment)
Frontend URL	`localhost:5173`
Backend URL	`localhost:5000`
Backend API base	`localhost:5000/api`
Database	PostgreSQL
Expected DB host	`localhost`
Expected DB name	`lifesync`
Expected DB user	`pattern`
Browser for automation	Google Chrome
Selenium mode	Headless by default, visible mode supported with `SELENIUM_HEADLESS=false`

Tooling

Tool	Version / Notes
Node test runner	`node --test` via `npm test`
Frontend build	`vite build`
Python	Python 3.13 local runtime
Selenium	`selenium 4.43.0`
Chrome observed in run	`chrome=142.0.7444.176`

4. Related Test And Related Requirement List

> Note: the repository does not expose an official editable requirement register with stable requirement IDs.
> The requirement IDs below are **working IDs derived from implemented behavior** in code and `docs/architecture/SDD_v3.md`.
> If you have official requirement IDs from your SRS or QA template, replace the IDs in the first column.

Test ID	Test / Check	Related Requirement ID	Related Requirement Description	Source	Result
T-AUTH-01	Register rejects missing required fields	REQ-AUTH-01	User registration must require e-mail, first name, last name, and password	Auth controller + API design	Pass
T-AUTH-02	Register rejects duplicate e-mail	REQ-AUTH-02	System must prevent duplicate account creation with the same e-mail	Auth controller + users table unique e-mail	Pass
T-AUTH-03	Login rejects invalid password	REQ-AUTH-03	System must authenticate only valid e-mail/password combinations	Auth/login flow in SDD	Pass
T-AUTH-04	Logout blacklists bearer token	REQ-AUTH-04	Logged-out tokens must not remain valid for protected routes	Auth controller + middleware	Pass
T-AUTH-05	Auth middleware validates JWT and injects user ID	REQ-AUTH-05	Protected endpoints must require a valid JWT and identify the user	Middleware + protected route design	Pass
T-PROFILE-01	ProfileBuilder validates password, age, gender, height, and weight	REQ-PROFILE-01	Profile updates must validate user-provided body data	Profile update flow in SDD	Pass
T-PROFILE-02	SqlUpdateBuilder preserves field order and null handling	REQ-PROFILE-02	Profile updates must build safe dynamic SQL updates	Builder pattern design	Pass
T-NOTIF-01	UserNotificationObserver writes notification records to DB	REQ-NOTIF-01	User actions must create notification records	Observer pattern implementation	Pass
T-PLAN-01	Prompt builder returns correct periods and row structure	REQ-PLAN-01	Diet and exercise prompts must use structured period-based plan output	Prompt builder utility	Pass
T-PLAN-02	Plan validator normalizes and sanitizes model output	REQ-PLAN-02	AI plan output must be sanitized and validated before use	Plan validator utility	Pass
T-E2E-01	Selenium smoke flow covers register, dashboard, profile edit, logout, and re-login	REQ-E2E-01	Main authentication and profile flow must work end-to-end in the UI	Frontend + backend integrated behavior	Pass

5. Test Inputs

Backend Unit Test Inputs

The backend unit test suite covers:

- Auth controller validation for missing required fields
- Duplicate e-mail registration checks
- Invalid login password handling
- Logout token blacklisting behavior
- Protected route token validation
- Malformed JWT rejection
- `ProfileBuilder` numeric range validation
- `ProfileBuilder` password and gender validation
- `SqlUpdateBuilder` field ordering and null handling
- Existing observer, prompt builder, validator, and DB integration scenarios already present in the repository

Selenium Smoke Test Inputs

The automated Selenium flow used the following runtime-generated test data pattern:

Input	Value
Email	`selenium_<random8>@example.com`
Password	`Test1234!`
First name	`Selenium`
Last name	`User<random4>`
Gender	`male`
Age	`27`
Height	`178`
Initial weight	`74`
Updated weight during profile edit	`76`

Commands Executed

```
cd backend
npm test
```

```
cd frontend
npm run build
```

```
cd client
python selenium_e2e.py
```

6. Test Results

Summary

Test Set	Result
Backend unit tests	Passed
Frontend build validation	Passed
Selenium smoke test	Passed
Test artifact recording	Completed

Detailed Results

Test Set	Measured Result	Outcome
Backend unit tests	`41/41` tests passed on 2026-04-16	Pass
Frontend build	Build completed successfully	Pass
Selenium smoke test	Registration, dashboard load, profile update, logout, and re-login completed successfully on 2026-04-16	Pass
Artifact recording	Logs and success screenshots saved under `docs/test-artifacts`	Pass

Coverage Improvement Delivered In This Cycle

- Added auth middleware unit tests
- Added auth controller unit tests
- Expanded builder test cases for null and invalid numeric inputs
- Confirmed the existing backend suite still passes after additions
- Confirmed end-to-end UI smoke flow with Selenium

Notes and Observations

- Backend test execution required the local environment because the sandbox blocked the Node test runner child process.
- Selenium ran successfully after making the UI more testable with stable `data-testid` selectors.
- Chrome UI may not appear unless `SELENIUM_HEADLESS=false` is set explicitly.

7. Recording Of The Test Results

Recorded Evidence Table

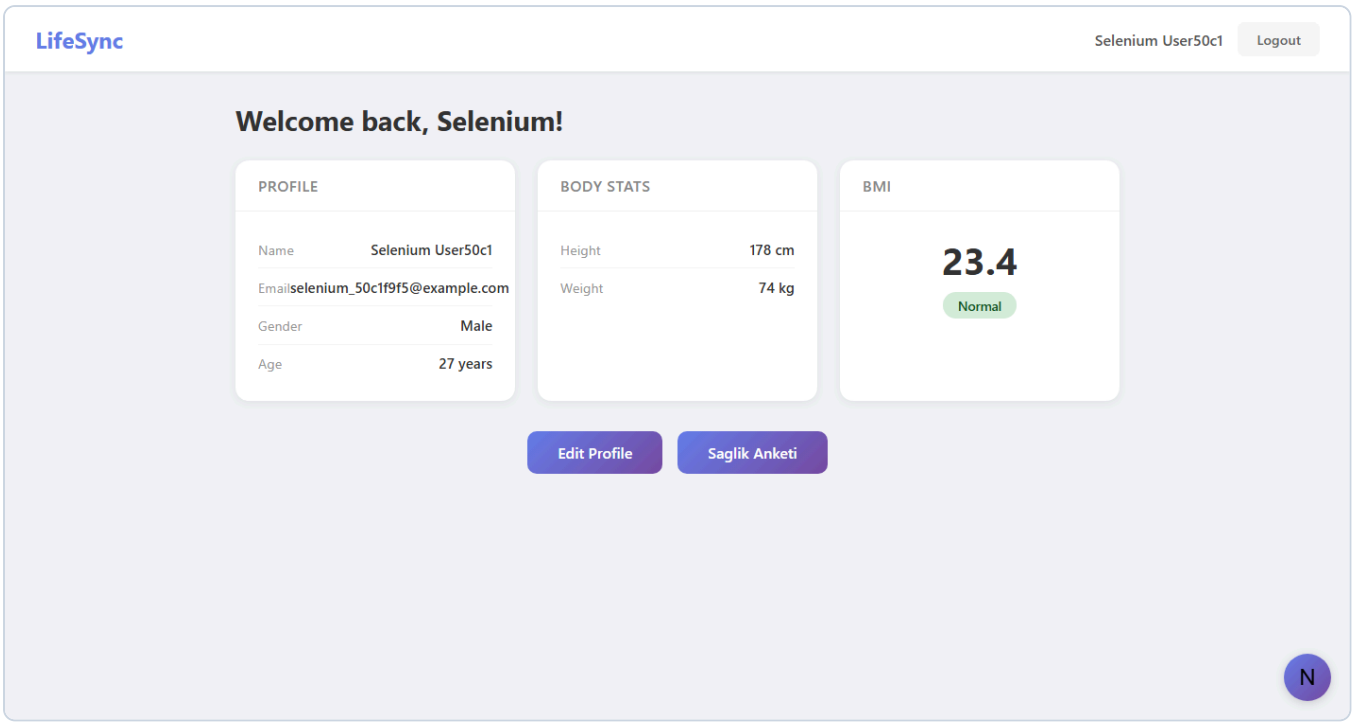
Evidence Type	File	Purpose	Status
Backend test log	`docs/test-artifacts/backend-test-output-2026-04-16-latest.txt`	Raw `npm test` log artifact for the latest successful run	Available
Frontend build log	`docs/test-artifacts/frontend-build-output-2026-04-16.txt`	Raw `npm run build` output for the latest successful run	Available
Backend dev log	`docs/test-artifacts/backend-dev-latest.log`	Backend startup log used for the latest Selenium run	Available
Frontend dev log	`docs/test-artifacts/frontend-dev-latest.log`	Frontend startup log used for the latest Selenium run	Available
Selenium output log	`docs/test-artifacts/selenium-e2e-output-2026-04-16.txt`	Raw Selenium smoke test result	Available
Screenshot	`docs/test-artifacts/01-login-page.png`	Login page evidence	Available
Screenshot	`docs/test-artifacts/02-signup-page.png`	Sign-up page evidence	Available
Screenshot	`docs/test-artifacts/03-dashboard-after-register.png`	Dashboard after successful registration	Available
Screenshot	`docs/test-artifacts/04-edit-profile.png`	Edit profile page evidence	Available
Screenshot	`docs/test-artifacts/05-profile-update-success.png`	Successful profile update evidence	Available
Screenshot	`docs/test-artifacts/06-dashboard-after-update.png`	Dashboard after profile update	Available
Screenshot	`docs/test-artifacts/07-dashboard-after-login.png`	Dashboard after re-login	Available

Embedded Screenshot Evidence

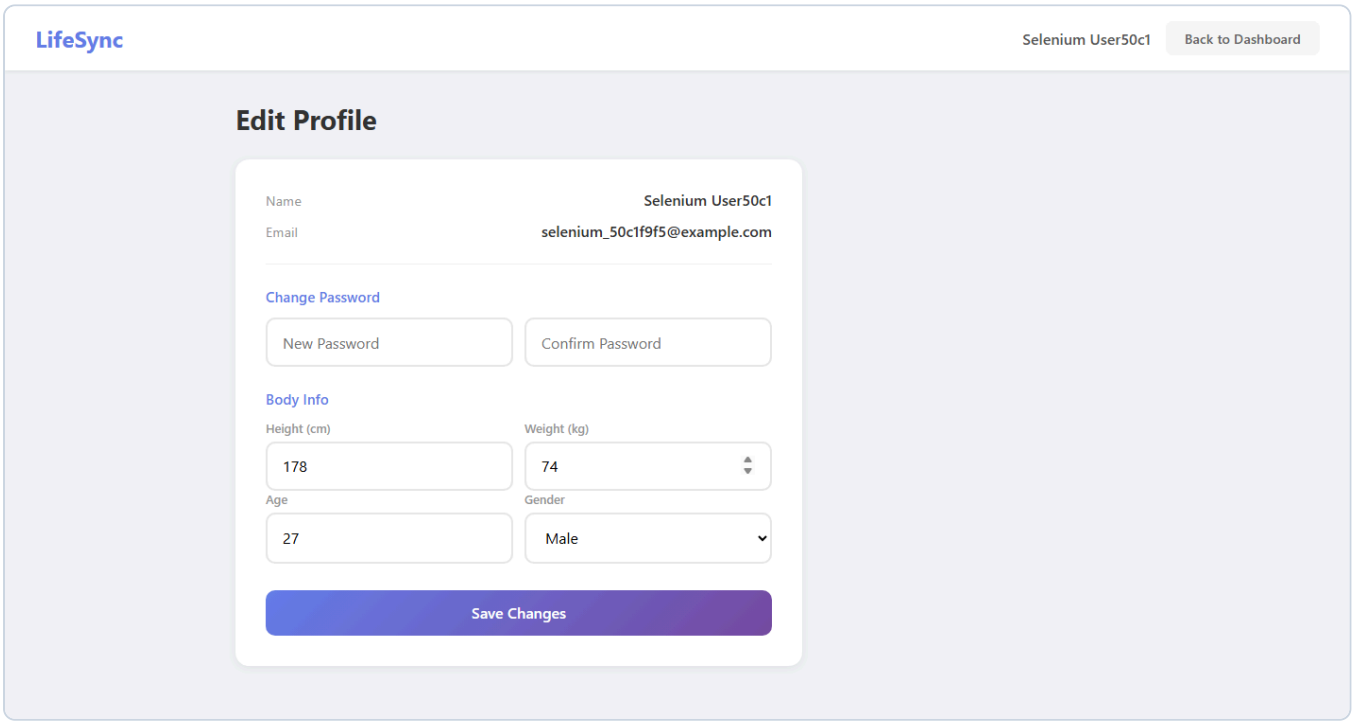
Login Page

Sign-Up Page

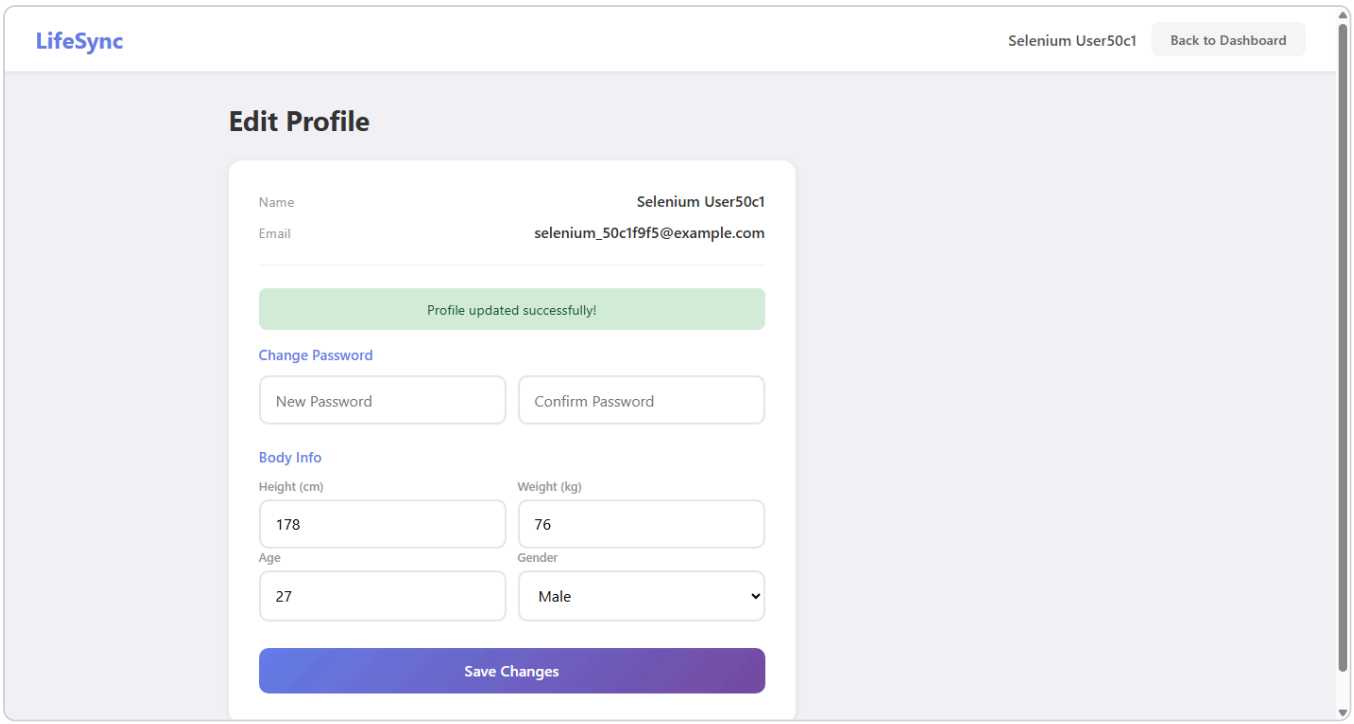
Dashboard After Registration



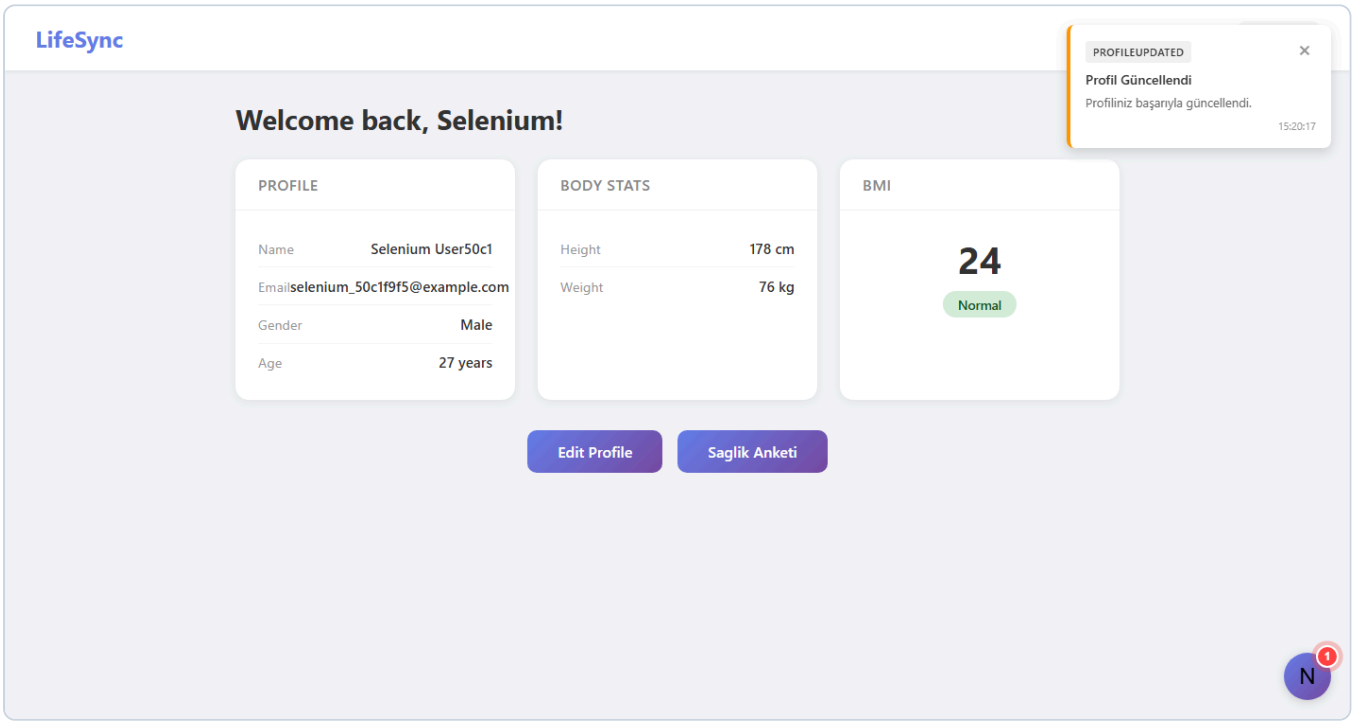
Edit Profile Page



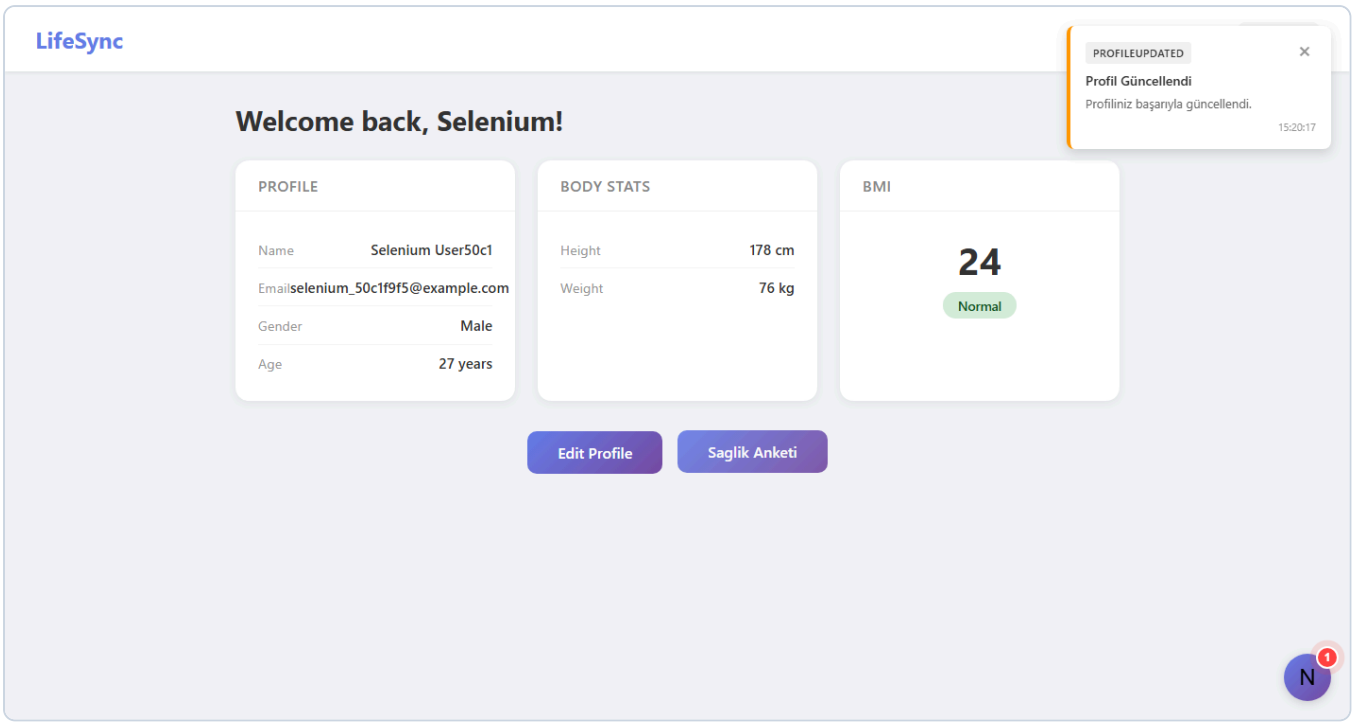
Profile Update Success



Dashboard After Update



Dashboard After Re-Login



Raw Backend Test Output

The full raw backend test output is stored in:

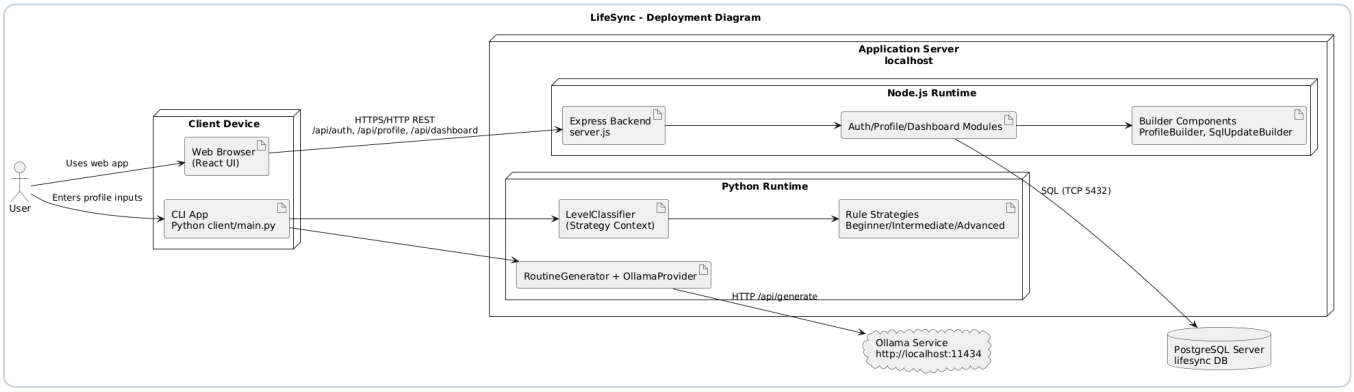
- `docs/test-artifacts/backend-test-output-2026-04-16-latest.txt`

The latest validated successful run summary used in this report is:

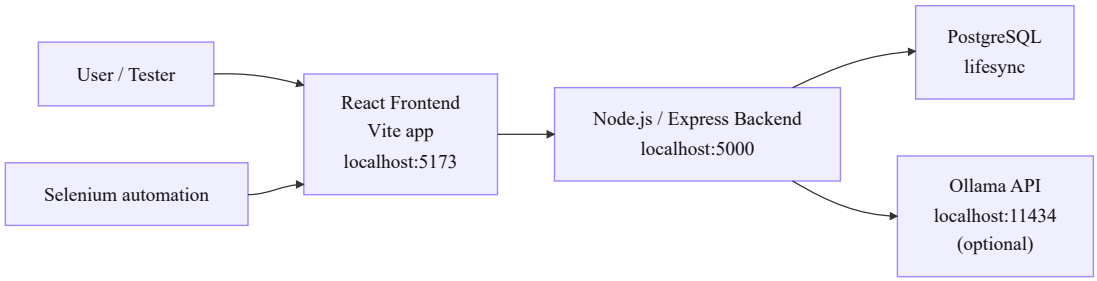
```
npm test
41 tests passed
0 failed
```

8. Deployment Diagram

Related Deployment Diagram



Mermaid Diagram



9. Design Patterns Verified By Tests

Pattern	Components Tested	Verification Scope
Facade Pattern	`WellnessPlanFacade`, dashboard survey and plan generation flow	Verified that Ollama-based survey and plan generation complexity is handled behind a simplified interface, with validation and internal processing hidden from the caller
Builder Pattern	`ProfileBuilder`, `SqlUpdateBuilder`	Verified that user-related update payloads can be built incrementally, validated safely, and converted into dynamic update statements
Observer Pattern	`Subject`, `Observer`, `PlanEventEmitter`, `UserNotificationObserver`	Verified that users are notified after actions such as survey completion and profile edit through the notification system
Factory Pattern	Export flow for report/result generation as PDF or Excel	Verified that exportable output can be produced in multiple formats from the same result domain
Strategy Pattern	User classification and personalized recommendation / plan selection behavior	Verified that behavior changes according to user level or selected generation path, enabling different recommendation or planning strategies

Pattern Verification Notes

- The tests validate the **Facade Pattern** by confirming that survey and plan-generation operations are exposed through a simplified entry point while Ollama calls, validation, and internal orchestration remain hidden behind the facade layer.
- The tests validate the **Builder Pattern** by checking that user-related data can be constructed step by step, validated, sanitized, and transformed into safe update payloads.
- The tests validate the **Observer Pattern** by confirming that user-triggered actions such as survey completion and profile editing generate notification events and downstream observer behavior.
- The tests validate the **Factory Pattern** at the feature level by confirming that result data can be exported in multiple output formats such as Excel and PDF.
- The tests validate the **Strategy Pattern** by confirming that recommendation and plan-generation behavior can vary based on user state, classification level, or selected planning path.

10. Open Items For Manual Completion

The following items could not be finalized with authoritative project data from the editable repository and may need your input:

Missing / Needs Confirmation	Why	What you can fill in
Official requirement IDs	The repository does not expose a plain-text requirement register with stable IDs	Replace `REQ-*` placeholders with your official SRS IDs
Named human test responsibles	Repo authors exist, but task ownership is not explicitly mapped in docs	Replace role-based responsibles with actual team members
Formal sign-off fields	Not present in current markdown source	Add approver name, role, and signature/date if required by your template

11. Conclusion

The current test cycle is **successful** based on the latest verified backend test run, successful frontend build, and successful Selenium smoke execution on 2026-04-16. The report now contains the requested test responsibles, test date, test configuration, test inputs, test results, related test-to-requirement mapping, recorded evidence references, and the related deployment diagram. The only remaining manual gaps are official requirement IDs, named human ownership, and any formal sign-off fields required by your template.